

Unit 5: Advance Topics on CSS

Prepared By:
Bipin Maharjan

Learning Objectives

- Develop advanced proficiency in CSS, covering flexbox, grid layouts, transitions, animations, responsiveness, specificity, units, preprocessors, and optimization.
- Master CSS Flexbox and Grid for versatile layout control.
- Apply transitions and animations using various properties and keyframes.
- Create responsive designs using media queries, breakpoints, and the mobile-first approach.
- Understand CSS specificity and inheritance principles for effective styling.
- Utilize different CSS units and values, including relative and absolute units.
- Explore CSS preprocessors, employing nesting, variables, mixins, and functions.
- Implement CSS best practices, including efficient organization, vendor prefixes, and optimization techniques.
- Enhance web development skills with advanced CSS techniques.

Table of Contents

- CSS Flexbox
- CSS Grid
- CSS Transitions and Animations
- Responsive Web Design
- CSS Specificity and Inheritance
- CSS Units and Values
- CSS Preprocessors
- CSS Best Practices and Optimization

CSS Flexbox

CSS Flexbox Overview

CSS Flexbox (Flexible Box Layout) is a one-dimensional layout system used to align and distribute space among items in a row or a column.

- One-dimensional → controls layout either horizontally or vertically
- Works on a parent–child relationship
- Ideal for navigation bars, cards, and simple layouts

Why Flexbox is Needed

Traditional layouts using float and position:

- Are difficult to align vertically
- Break easily on different screen sizes

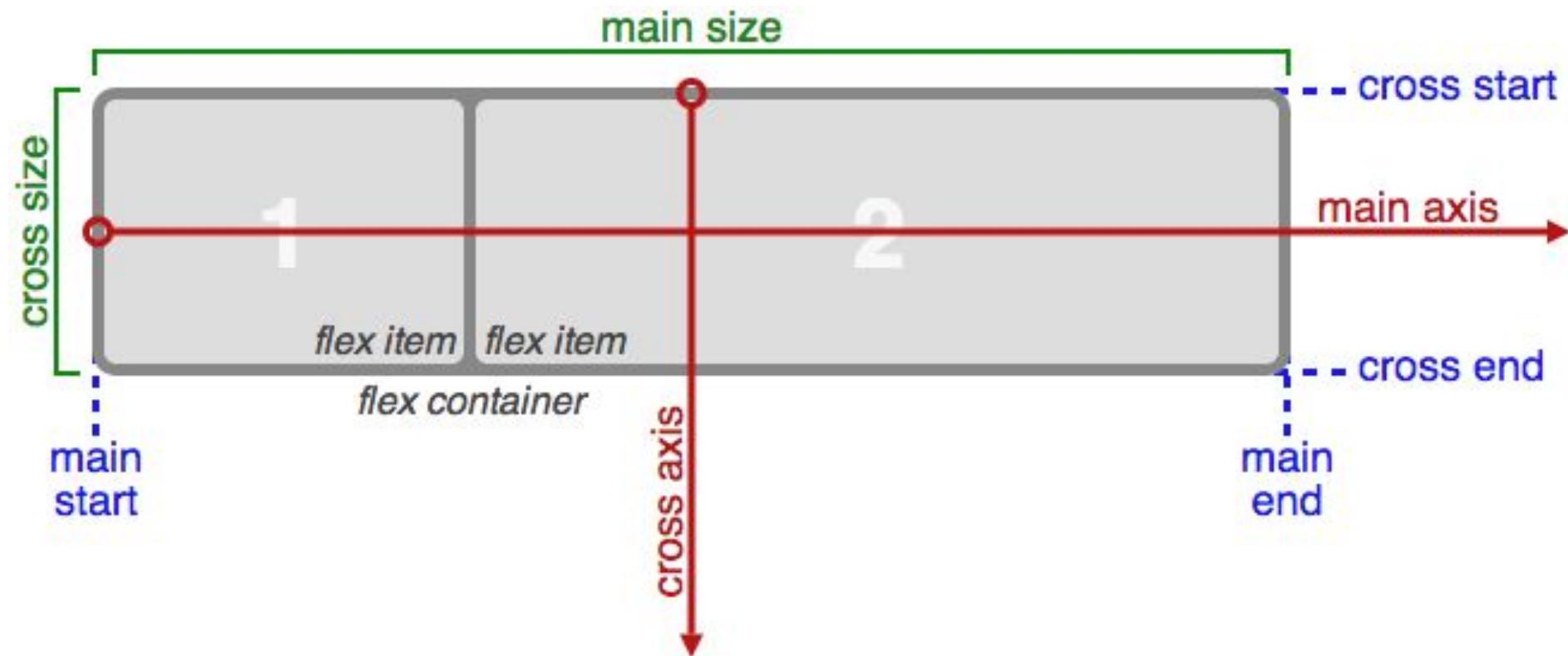
Flexbox solves:

- Vertical and horizontal centering
- Equal-height columns
- Flexible spacing

CSS Flexbox Components

A flexbox always consists of:

- **A Flex Container** - The parent (container) element, where the display property is set to flex or inline-flex
- **One or more Flex Items** - The direct children of the flex container automatically becomes flex items



Flex Container Properties

```
.container {  
  
  display: flex;  
  
  flex-direction: row;  
  
  justify-content: space-between;  
  
  align-items: center;  
  
  flex-wrap: nowrap;  
  
}
```

Property	Purpose
<code>display: flex</code>	Enables flexbox
<code>flex-direction</code>	Row, column, row-reverse, column-reverse
<code>justify-content</code>	Aligns along main axis
<code>align-items</code>	Aligns along cross axis
<code>flex-wrap</code>	nowrap(default), wrap, wrap-reverse

1. flex-direction

```
.container {  
  flex-direction: row;  
}
```

Value	Description
row	Left → Right
row-reverse	Right → Left
column	Top → Bottom
column-reverse	Bottom → Top

2. justify-content (Main Axis Alignment)

```
.container {  
  justify-content: space-between;  
}
```

Value	Effect
<code>flex-start</code>	Items at start
<code>flex-end</code>	Items at end
<code>center</code>	Items at center
<code>space-between</code>	Equal space between
<code>space-around</code>	Space around items
<code>space-evenly</code>	Equal space everywhere

3. align-items (Cross Axis Alignment)

```
.container {
```

```
  align-items: center;
```

```
}
```

Value	Description
stretch	Default, stretches items
center	Center vertically
flex-start	Top
flex-end	Bottom
baseline	Align text baseline

4. flex-wrap

```
.container {  
  flex-wrap: wrap;  
}
```

Value	Meaning
nowrap	Default, no wrapping
wrap	Items move to next line
wrap-reverse	Reverse wrapping

5. gap

Adds space between items.

```
.container {  
  gap: 20px;  
}
```

Flex Item Properties

```
.item {  
  flex: 1 1 200px;  
}
```

Expanded form

```
.item {  
  flex-grow: 1;  
  flex-shrink: 1;  
  flex-basis: 200px;  
}
```

Property	Purpose
<code>flex-grow</code>	How much item grows
<code>flex-shrink</code>	How much item shrinks
<code>flex-basis</code>	Initial size

Example 1: Simple Horizontal Layout

```
<div class="container">
```

```
  <div class="box">Home</div>
```

```
  <div class="box">About</div>
```

```
  <div class="box">Contact</div>
```

```
</div>
```

```
.container {
```

```
  display: flex;
```

```
  gap: 20px;
```

```
  background-color: lightgray;
```

```
}
```

```
.box {
```

```
  background: lightblue;
```

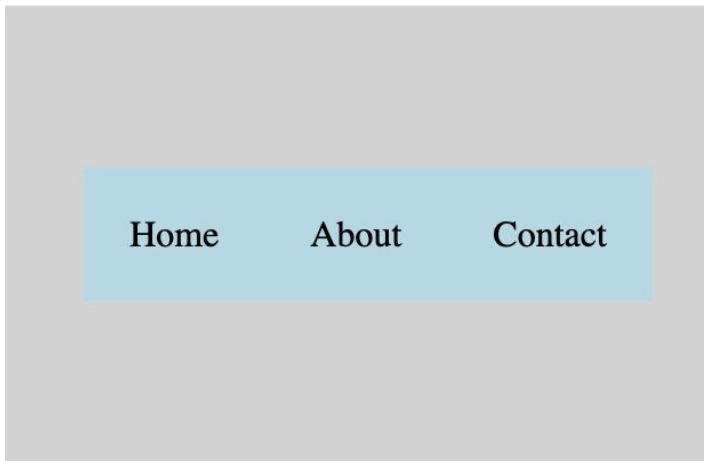
```
  padding: 20px;
```

```
}
```



Example 2: Centering Elements

```
<div class="container">  
  <div class="box">Home</div>  
  <div class="box">About</div>  
  <div class="box">Contact</div>  
</div>
```



```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 200px;  
}  
  
.box {  
  background: lightblue;  
  padding: 20px;  
}
```

Example 3: Equal Width Columns

```
<div class="container">  
  <div class="box">Home</div>  
  <div class="box">About</div>  
  <div class="box">Contact</div>  
</div>
```

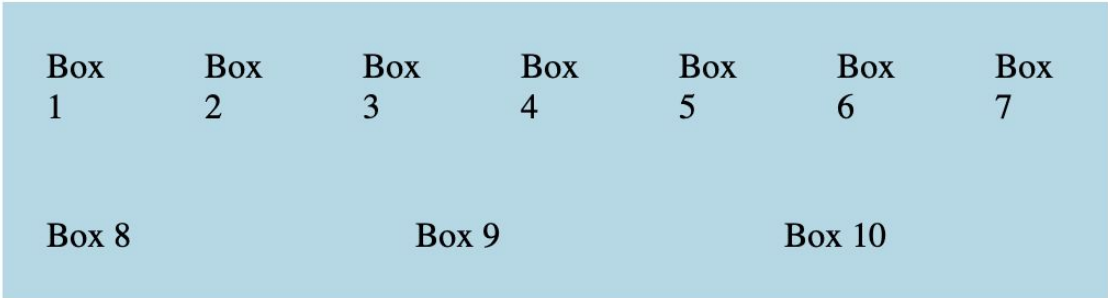


```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 200px;  
}  
  
.box {  
  flex: 1;  
}
```

Example 4: Wrapping Items

```
<div class="container">  
  <div class="box">Box 1</div>  
  <div class="box">Box 2</div>  
  <div class="box">Box 3</div>  
  <div class="box">Box 4</div>  
  <div class="box">Box 5</div>  
  <div class="box">Box 6</div>  
  <div class="box">Box 7</div>  
  <div class="box">Box 8</div>  
  <div class="box">Box 9</div>  
  <div class="box">Box 10</div>  
</div>
```

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
}  
.box {  
  flex: 1;  
  background: lightblue;  
  padding: 20px;  
}
```



Box 1	Box 2	Box 3	Box 4	Box 5	Box 6	Box 7
Box 8	Box 9	Box 10				

CSS Grid

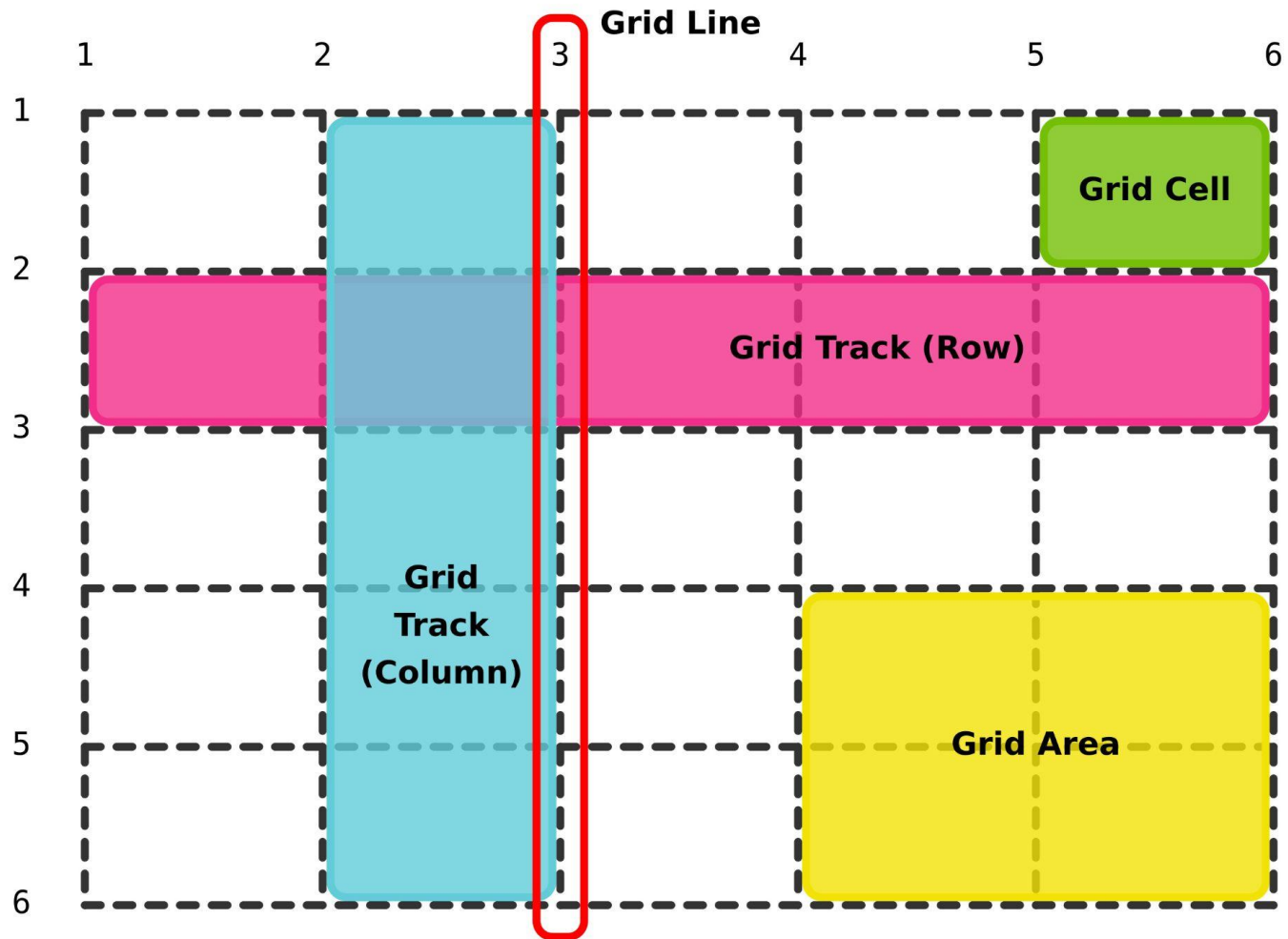
What is CSS Grid?

- CSS Grid Layout is a two-dimensional layout system that allows developers to control rows and columns simultaneously.
- Unlike Flexbox (one-dimensional), Grid is designed for:
 - Full-page layouts
 - Complex structures
 - Precise placement of elements
- When display: grid is applied to a parent element, it becomes a grid container, and its direct children become grid items.

CSS Grid Components

A grid always consists of:

- **A Grid Container** - The parent (container) element, where the display property is set to grid or inline-grid
- **One or more Grid Items** - The direct children of the grid container automatically becomes grid items



Core Grid Concepts

Term	Meaning
Grid Container	Parent element with <code>display: grid</code>
Grid Item	Direct child of grid container
Grid Line	Dividing lines between rows/columns
Grid Cell	Single grid unit
Grid Area	Rectangular group of cells

Creating a Grid Container

The grid container is the parent of all grid items. It is declared with the `display:grid;` rule.

```
.container {  
    display: grid;  
}
```

Grid Item

The items that are direct children of the grid container.

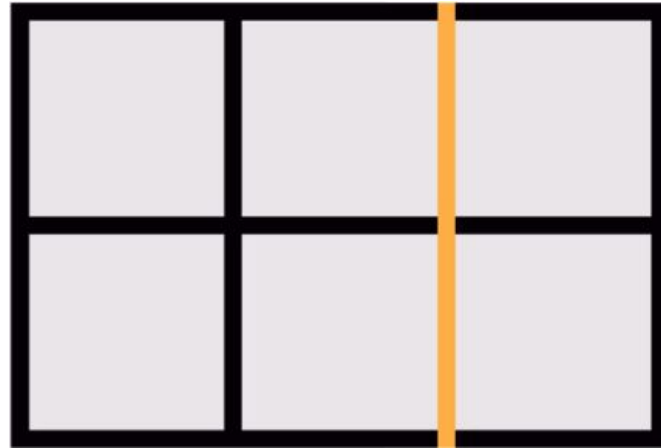
These are the elements that will be placed on the grid.

```
<div class="grid-container">  
  <div class="item-a"></div>  
  <div class="item-b"></div>  
  <div class="item-c"></div>  
</div>
```

Grid Lines

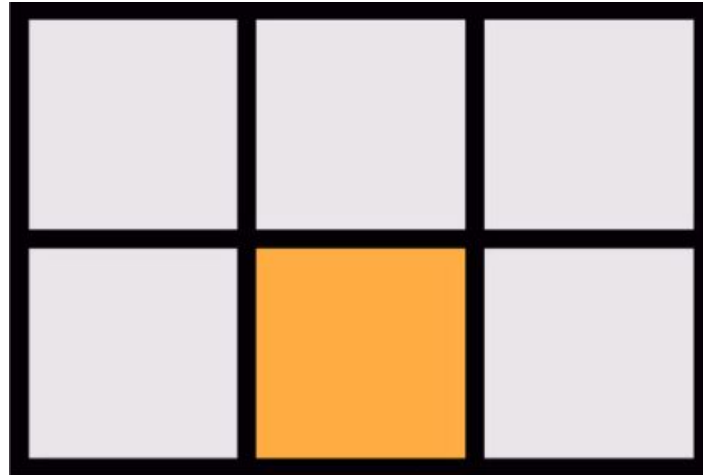
The lines that divide the columns and rows of the grid.

There are numbered starting with 1.



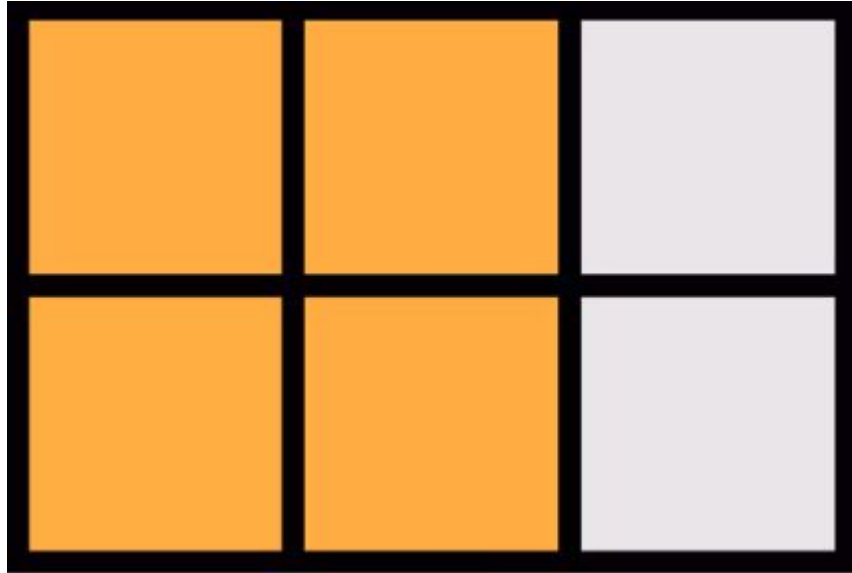
Grid Cell

The space between 2 adjacent grid column lines and 2 adjacent grid row lines.



Grid Area

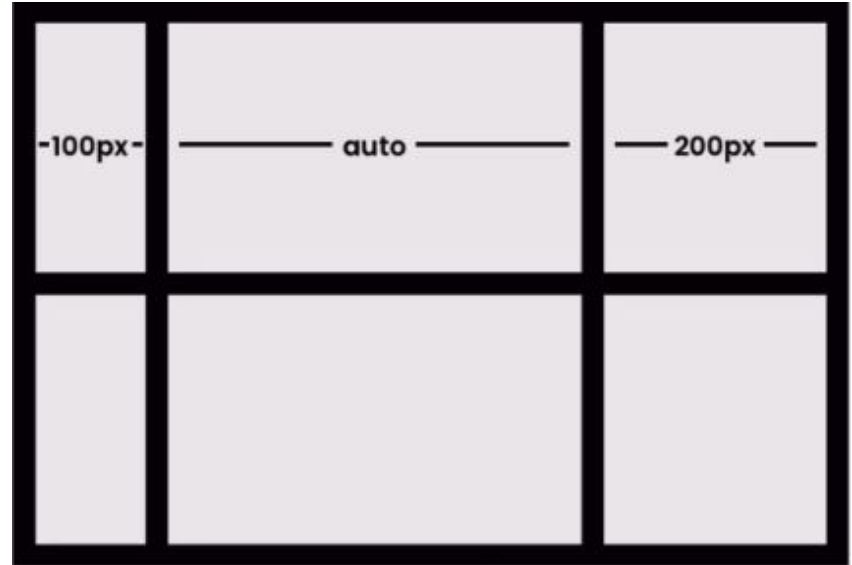
The space between any 4 grid lines. It can cover any number of rows and columns.



Grid Template Columns

Defines the width of the columns in the grid.

`grid-template-columns: 100px auto 200px;`

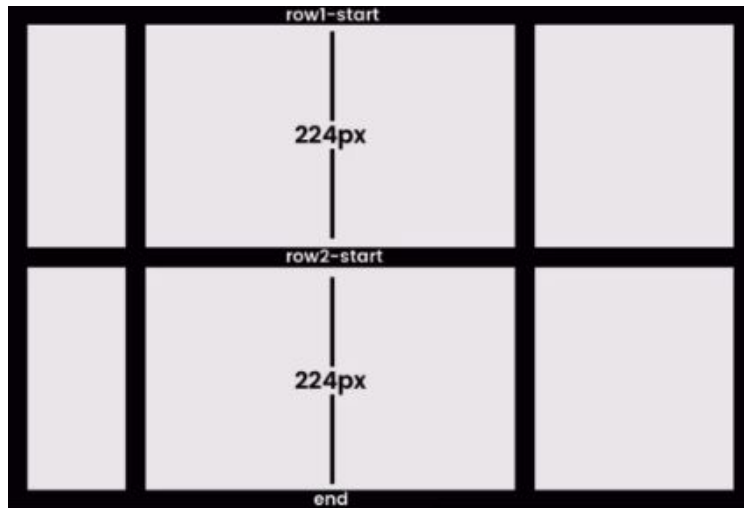


Grid Template Rows

Defines the height of the grid rows.

```
grid-template-rows: 224px 224px;
```

```
grid-template-rows: [row1-start] [row2-start]  
[end];
```



Grid Gap

grid-column-gap

Define the size of the grid lines between columns.

```
grid-column-gap: 30px;
```

grid-row-gap

Defines the size of the grid lines between rows.

```
grid-row-gap: 40px;
```

Grid Gap

Shorthand for row and column gap.

Grid-gap: <grid-row-gap> <grid-column-gap>;

grid-gap: 40px 30px;

Note: The gap is only between columns and rows.

Not outside.



Grid Template Area

Allows you to name sections of the grid.

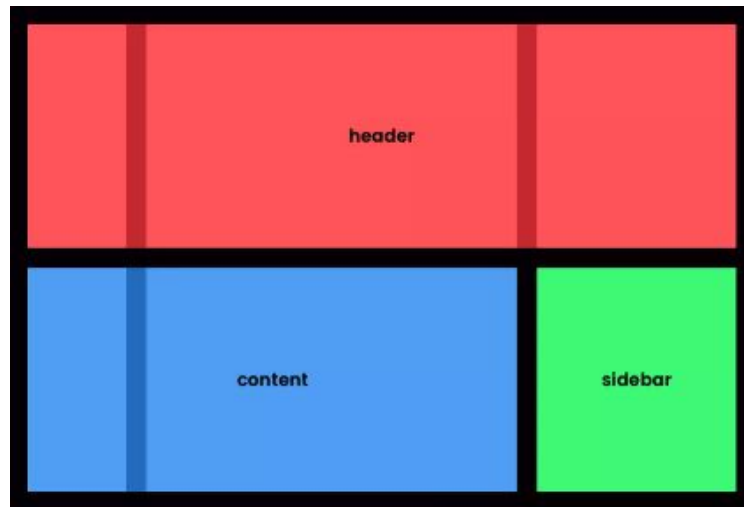
`grid-template-columns: 100px auto 200px;`

`grid-template-rows: [row1-start] 224px [row2-start] 224px [end];`

`grid-template-areas:`

`“header header header”`

`“content content sidebar”`



Grid Template Area Example

HTML

```
<div class="layout">

  <header>Header</header>

  <nav>Menu</nav>

  <main>Content</main>

  <footer>Footer</footer>

</div>
```

CSS

```
.layout {
  display: grid;
  grid-template-areas:
    "header header"
    "menu content"
    "footer footer";
  grid-template-columns: 1fr 3fr;
  gap: 10px;
}

header { grid-area: header; }
nav { grid-area: menu; }
main { grid-area: content; }
footer { grid-area: footer; }
```

Fractional (FR) Units

Grid comes with the fr unit. Which uses up a fraction of the remaining free space in the grid.

```
grid-template-columns: 1fr 2fr 1fr;
```

Repeat() Notation

Allows you to easily repeat settings.

```
grid-template-columns: repeat(12, 1fr);
```

Grid Column Start

Grid Column Start / Grid Column End | Grid Column

grid-column-start: 2;

grid-column-end: 4;

Shorthand:

grid-column: 2/4;

Grid Row Start

Grid Row Start / Grid Row End | Grid Row

grid-row-start: 1;

grid-row-end: 2;

Shorthand

grid-row: 1/2;

Placing Grid Items

```
.item-a {  
  grid-column: 2/4;  
  grid-row: 1/2;  
}
```

```
.item-b {  
  grid-column: 1/2;  
  grid-row: 1/3;  
}
```

```
.item-c {  
  grid-column: 2/4;  
  grid-row: 2/3;  
}
```



Website Page Layout: Example

```
<div class="page">
```

```
  <header>Header</header>
```

```
  <aside>Sidebar</aside>
```

```
  <section>Main Content</section>
```

```
  <footer>Footer</footer>
```

```
</div>
```

```
.page {  
  display: grid;  
  grid-template-columns: 1fr 3fr;  
  grid-template-rows: 80px auto 60px;  
  gap: 10px;  
}
```

```
header {  
  grid-column: 1 / 3;  
  background: #4caf50;  
}
```

```
aside {  
  background: #f4f4f4;  
}
```

```
section {  
  background: #ddd;  
}
```

```
footer {  
  grid-column: 1 / 3;  
  background: #333;  
  color: white;  
}
```

Image Gallery Layout: Example

```
<div class="gallery">  
  <div>Image 1</div>  
  <div>Image 2</div>  
  <div>Image 3</div>  
  <div>Image 4</div>  
</div>
```

```
.gallery {  
  display: grid;  
  grid-template-columns: repeat(auto-fit,  
    minmax(150px, 1fr));  
  gap: 15px;  
}  
  
.gallery div {  
  background: lightgray;  
  padding: 40px;  
  text-align: center;  
}
```

Dashboard Layout Using Grid Areas

```
<div class="dashboard">  
  <div class="top">Top Bar</div>  
  <div class="menu">Menu</div>  
  <div class="content">Content</div>  
  <div class="ads">Ads</div>  
</div>
```

```
.dashboard {  
  display: grid;  
  grid-template-areas:  
    "top top top"  
    "menu content ads";  
  grid-template-columns: 1fr 3fr 1fr;  
  gap: 10px;  
}  
  
.top { grid-area: top; background: #2196f3; }  
.menu { grid-area: menu; background: #ccc; }  
.content { grid-area: content; background: #eee; }  
}  
.ads { grid-area: ads; background: #ddd; }
```

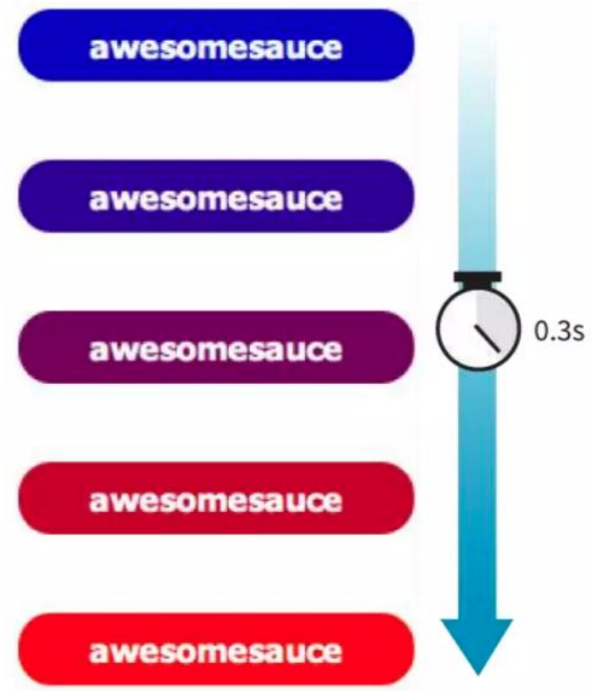
CSS Transitions and Animations

CSS Transitions

- CSS transitions create a smooth change from one state to another.
- They fill in the frames in between (tweening).
- Example: Gradually changing a button from red to blue (through purple) when the mouse pointer hovers over it.

State 1: Default

State 2: When the mouse is over the element



Transition Properties

- **transition-property**
 - Which CSS property to change
- **transition-duration**
 - How long the transition should take in seconds (or milliseconds)
- **transition-timing-function**
 - The manner in which the transition accelerates
- **transition-delay**
 - Whether there should be a pause before the transition starts and how long that pause should be (in seconds)

transition-property

Values: Property-name, all, none

Identifies the property that will receive a transition when it changes state.

Here, we want to smooth out the change in background color when the color changes from hovering or focus:

```
.smooth {  
    ...  
    color: #fff;  
    background-color: mediumblue;  
    transition-property: background-color;  
}  
.smooth: hover, .smooth:focus {  
    background-color: red;  
}
```

transition-duration

Values: Time

Identifies how much time the transition will take. It's usually specified in second (s) or milliseconds (ms).

In this example, the transition from blue to red takes .3 seconds:

```
.smooth {  
    ...  
    color: #fff;  
    background-color: mediumblue;  
    transition-property: background-color;  
    transition-duration: .3s;  
}  
.smooth: hover, .smooth:focus {  
    background-color: red;  
}
```

transition-timing-function

Values: ease, linear, ease-in, ease-out, ease-in-out, step-start, step-end, steps, cubic-bezier (#, #, #, #)

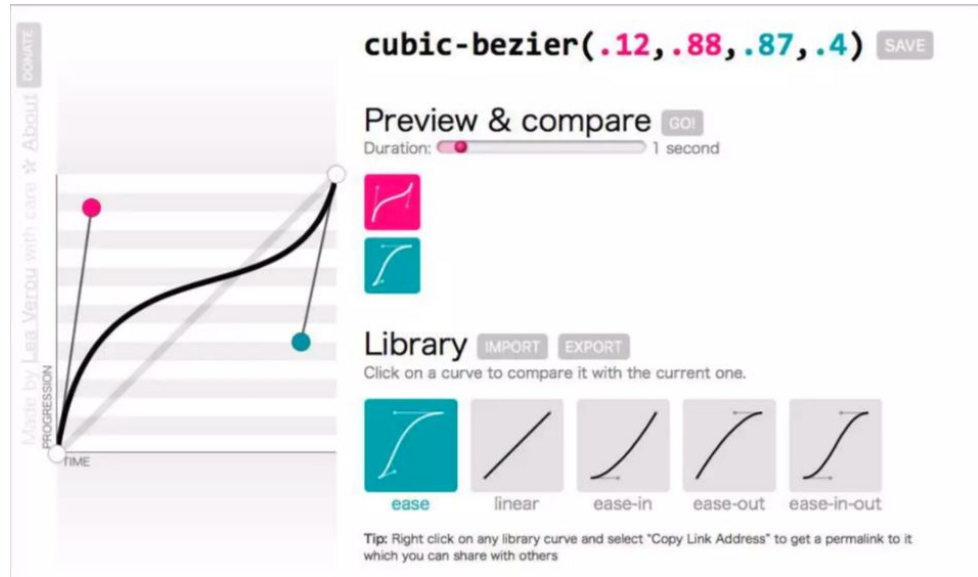
- The **timing function** describes the way the transition accelerates or decelerates over time.
- It has a big impact on the feel and believability of the animation.
- The default is **ease**, which starts slowly, accelerates quickly, then slows down again at the end.

Timing functions (cont'd)

- **linear**: Stays consistent from beginning to end, feels mechanical
- **ease-in**: starts slowly, then speeds up
- **ease-out**: starts quickly, then slows down
- **ease-in-out**: Similar to ease, but with less acceleration in the middle
- **cubic-bezier (#, #, #, #)**: Defines a curve that plots acceleration
- **steps (#, start or end)**: Divides the animation into a number of steps. The start and end keywords indicate whether that transition happens at the beginning or end of each step.
- **step-start**: Changes states in one step, at the beginning of the duration time
- **step-end**: Changes states in one step, at the end of the duration time

Cubic Bezier Curves

- Acceleration can be plotted using a Bezier curve
- Steep sections indicate quick rate of change; flat parts indicate slow rate of change
- The curve is defined by the x,y coordinates of “handles” that control the curve.



transition-delay

Values: Time

Delays the start of the transition by the amount of time specified.

In this example, the transition will begin .2 seconds after the user hovers over the element:

```
.smooth {  
    ...  
    color: #fff;  
    background-color: mediumblue;  
    transition-property: background-color;  
    transition-duration: .3s;  
    transition-timing-function: ease-in-out;  
    transition-delay: 0.2s;  
}  
.smooth: hover, .smooth:focus {  
    background-color: red;  
}
```

Shorthand transition Property

Values: property duration timing-function delay

Combines all the transition properties into one declaration. Values are separated by character spaces.

The duration time must appear before delay time.

```
.smooth {  
    ...  
    color: #fff;  
    background-color: mediumblue;  
    transition: background-color .3s ease-in-out 0.2s;  
}
```

Example: Expanding Box on Hover

HTML

```
<div class="box"></div>
```

CSS

```
.box {  
  width: 100px;  
  height: 100px;  
  background: blue;  
  transition: width 0.5s ease;  
}
```

```
.box:hover {  
  width: 200px;  
}
```

Example: Image Zoom Effect

HTML

```

```

CSS

```
.img {  
    width: 200px;  
    transition: transform 0.4s ease;  
}
```

```
.img:hover {  
    transform: scale(1.1);  
}
```

Example: Button Shadow Effect

HTML

```
<button class="btn">Click</button>
```

CSS

```
.btn {  
  padding: 10px 20px;  
  background: green;  
  color: white;  
  border: none;  
  transition: box-shadow 0.3s ease;  
}  
  
.btn:hover {  
  box-shadow: 0 4px 8px rgba(0,0,0,0.3);  
}
```

What is an animation?

- The state or condition of being animated
- Containing representations of animals or mechanical objects that appear to move as real one do
 - Dictionary.com
- A way for authors to animate the values of CSS properties over time, using keyframes. The behavior of these keyframe animations can be controlled by specifying their duration, number of repeats, and repeating behavior.
 - W3C Draft Specification

Intro to Keyframe Animation



Keyframe animation enables you to create transitions between a series of states (keyframes):

1. **Establish the keyframes** with a `@keyframes` rule:

```
@keyframes animation-name {  
    keyframe { property: value; }  
    /* additional keyframes */  
}
```

2. **Apply animation properties** to the element(s) that will be animated.

Animations CSS Properties

- **Animation name:** name of defined animation
- Animation **duration**, **timing function**, **delay** same as transition
- **Animation iteration count:** how many times it will loop
- **Animation direction:** direction and alternate cycles
- **Animation play state:** running or paused (can toggle)
- **Animation fill mode:** sets end state after animation

Must have a name and duration set for animations to work.

Cont'd

Keyframes establish colors at each point in the animation and give the sequence a name (“rainbow”):

```
@keyframes rainbow {  
  0% { background-color: red; }  
  20% { background-color: orange; }  
  40% { background-color: yellow; }  
  60% { background-color: green; }  
  80% { background-color: blue; }  
  100% { background-color: purple; }  
}
```

The animation properties are applied to the animated element (including which keyframe sequence to use):

```
#magic {  
  ...  
  animation-name: rainbow;  
  animation-duration: 5s;  
  animation-timing-function: linear;  
  animation-iteration-count: infinite;  
  animation-direction: alternate;  
}
```

Example 1: Moving Box (Left to Right)

HTML

```
<div class="move"></div>
```

CSS

```
.move {  
  width: 100px;  
  height: 100px;  
  background: red;  
  position: relative;  
  animation: slide 2s infinite;  
}
```

```
@keyframes slide {  
  from { left: 0; }  
  to { left: 200px; }  
}
```

Example 2: Fade In Effect

HTML

```
<div class="fade">Hello</div>
```

CSS

```
.fade {  
    animation: fadeIn 2s;  
}
```

```
@keyframes fadeIn {  
    from { opacity: 0; }  
    to { opacity: 1; }  
}
```

Example: Progress Bar Animation

HTML

```
<div class="progress">  
  <div class="bar"></div>  
</div>
```

CSS

```
.progress {  
  width: 300px;  
  height: 20px;  
  background: #ddd;  
}  
  
.bar {  
  height: 100%;  
  width: 0;  
  background: green;  
  animation: load 3s forwards;  
}  
  
@keyframes load {  
  to { width: 100%; }  
}
```

Responsive Web Design

What is Responsive Web Designing?

- Responsive Web Designing (RWD) is a process of designing a single website to be used and compatible on different portable or handy electronic devices.
- Also known as Adaptive Web Designing (AWD).
- It regarded as an integrated approach of designing through which compelling and easy to use websites are built, to give an optimal viewing user experience across a wide variety of devices starting from desktop computers to mobile phones.

Core Components of Responsive Design

1. Flexible layouts (Flexbox / Grid)
2. Media queries
3. Responsive units
4. Mobile-first approach

Example: Responsive Two-Column Layout

HTML

```
<div class="layout">

  <div class="box">Left</div>

  <div class="box">Right</div>

</div>
```

CSS

```
.layout {
  display: flex;
}

.box {
  flex: 1;
  padding: 20px;
  background: lightblue;
}

/* Mobile view */
@media (max-width: 600px) {
  .layout {
    flex-direction: column;
  }
}
```

Example: Responsive Navigation Menu

HTML

```
<nav class="nav">

  <a href="#">Home</a>

  <a href="#">About</a>

  <a href="#">Contact</a>

</nav>
```

CSS

```
.nav {
  display: flex;
  gap: 15px;
}

/* Mobile */
@media (max-width: 600px) {
  .nav {
    flex-direction: column;
  }
}
```

Example: Responsive Image Gallery

HTML

```
<div class="gallery">
  <div>Image 1</div>
  <div>Image 2</div>
  <div>Image 3</div>
</div>
```

CSS

```
.gallery {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}

/* Desktop */
@media (min-width: 992px) {
  .gallery {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

CSS Specificity and Inheritance

What is CSS Specificity?

CSS Specificity is a set of rules used by the browser to decide which CSS rule is applied when multiple rules target the same element.

Why Specificity is Needed?

- Multiple CSS rules can apply to the same element
- Browser must resolve conflicts
- Prevents ambiguity in styling

Making the decision, overview

Browsers take three things into account before applying CSS:

1. Source Order
 - a. Whichever ruleset comes last will be applied
2. Specificity
 - a. Whichever ruleset carries greater weight / more specific selectors will be applied
3. Importance
 - a. Whichever ruleset is marked as being more important will be applied

Specificity Hierarchy

From lowest to highest:

Selector Type	Example	Specificity Value
Universal	<code>*</code>	0,0,0,0
Element	<code>p, div</code>	0,0,0,1
Class / Attribute / Pseudo-class	<code>.box, [type], :hover</code>	0,0,1,0
ID	<code>#main</code>	0,1,0,0
Inline Style	<code>style=""</code>	1,0,0,0
<code>!important</code>	overrides all	Highest

Making the decision via specificity

The most specific selector: If more than one declaration applies to the same element, then the most specific selector wins

i.e. `h2.headline` will apply over `.headline`

This also works in overriding inherited styles, i.e., `p > .red` will override `p`.

The most weighted selector: The selector that has the highest specificity weight will win.

i.e. `#sidebar { ... }` weighs more than `aside { ... }`

Example: Specificity Conflict

HTML

```
<p id="text" class="para">Hello</p>
```

CSS

```
p {  
  color: blue;  
}
```

```
.para {  
  color: green;  
}
```

```
#text {  
  color: red;  
}
```

!important (Use Carefully)

```
p {
```

```
  color: blue !important;
```

```
}
```

- Overrides all selectors
- Should be avoided in best practices
- Acceptable only in exceptional cases

What is CSS Inheritance?

- Inheritance is the mechanism by which child elements automatically receive certain styles from their parent elements.
- Elements inherit styles from their parent container.
- Not all properties are inherited, but you can force ones to be by using the inherit value.
- For example

```
p { margin: inherit, padding: inherit; }
```

Properties already have “inherit” as a default value

- border-collapse
- border-spacing
- caption-side
- color
- cursor
- direction
- empty-cells
- font
- font-family
- font-stretch
- font-size
- font-size-adjust
- font-style
- font-variant
- font-weight
- letter-spacing
- line-height
- list-style
- list-style-image
- list-style-type
- quotes
- text-align
- text-indent
- text-transform
- white-space
- word-spacing

Example: Inheritance

HTML

```
<div class="parent">  
  <p>Child paragraph</p>  
</div>
```

CSS

```
.parent {  
  color: blue;  
  font-family: Arial;  
}
```

Example: Overriding Inherited Styles

HTML

```
<div class="parent">  
  <p>Child paragraph</p>  
</div>
```

CSS

```
.parent {  
  color: blue;  
  font-family: Arial;  
}  
  
p {  
  color: red;  
}
```

inherit, initial, and unset

inherit

Forces inheritance:

```
p { color: inherit; }
```

initial

Resets to default:

```
p { color: initial; }
```

unset

Acts as:

- inherit (if inheritable)
- initial (if not)

```
p {  
    color: unset;  
}
```

CSS Units and Values

What are CSS Units?

CSS units define the size, length, spacing, or measurement of elements such as width, height, font size, margin, padding, etc.

Example:

```
p {  
    font-size: 16px;  
}
```

Classification of CSS Units

CSS units are broadly classified into:

1. Absolute Units
2. Relative Units

Absolute Units

Absolute units have fixed values and do not change based on screen size.

Unit	Description
px	Pixels (most used)
cm	Centimeters
mm	Millimeters
in	Inches
pt	Points
pc	Picas

When to Use Absolute Units

- Borders
- Icons
- Small fixed UI elements
- When exact size is required

Relative Units

Relative units depend on another value, such as:

- Parent element
- Root element
- Viewport size

Relative Units Based on Font Size

em

- Relative to parent font size

```
.parent {  
  font-size: 20px;  
}  
  
.child {  
  font-size: 1.5em; /* 30px */  
}
```

rem

- Relative to root (html) font size

```
html {  
  font-size: 16px;  
}  
  
p {  
  font-size: 1.25rem; /* 20px */  
}
```

Relative Units Based on Container Size

%

- Based on parent element
- Used in layouts

```
.box {
```

```
  width: 50%;
```

```
}
```

Viewport Units

Unit	Meaning
vw	1% of viewport width
vh	1% of viewport height
vmin	Smaller of vw or vh
vmax	Larger of vw or vh

CSS Preprocessors

Whats is a preprocessor?

- A preprocessor is a program that takes one type of data and converts it to another type of data.
- It's used in development to write less CSS and keep your CSS as clean and readable as possible.

Why preprocessing your CSS?

- CSS by itself is primitive
- Avoid duplicate code
- Nested code
- Increase of productivity
- Helps to have system-wide definitions
- Ability to write functions and have variables
- Error reporting
- No worries with cross-browser compatibility

Main preprocessors

The Sass logo is written in a fluid, cursive script using a vibrant magenta color.The Less logo features the word "less" in a bold, white, sans-serif font, centered within a dark blue rectangular box with rounded corners. The box is flanked by large, white curly braces on both sides.The Stylus logo is rendered in a black, elegant script font, with a distinctive underline beneath the final letters.

Variables:

Variables store reusable values like colors, fonts, sizes.

```
$font-size: 16px;
```

```
div {  
  font-size: $font-size;  
}
```

Sass

```
@font-size: 16px;
```

```
div {  
  font-size: @font-size;  
}
```

Less

```
font-size = 16px
```

```
div  
  font-size font-size
```

Stylus

```
div {  
  font-size: 16px;  
}
```

Css

Nesting

Sass

```
$link-color: #999;  
$link-hover: #229ed3;  
  
ul {  
  margin: 0;  
  
  li {  
    float: left;  
  }  
  
  a {  
    color: $link-color;  
  
    &:hover {  
      color: $link-hover;  
    }  
  }  
}
```

Less

```
@link-color: #999;  
@link-hover: #229ed3;  
  
ul {  
  margin: 0;  
  
  li {  
    float: left;  
  }  
  
  a {  
    color: @link-color;  
  
    &:hover {  
      color: @link-hover;  
    }  
  }  
}
```

Stylus

```
link-color = #999  
link-hover = #229ed3  
  
ul  
  margin 0  
  li  
    float left  
  a  
    color link-color  
    &:hover  
      color link-hover
```

Css

```
ul { margin: 0; }  
ul li { float: left; }  
ul a { color: #999; }  
ul a:hover { color: #229ed3; }
```

Imports

Split CSS into smaller files and combine them.

```
@import "library";  
@import "mixins/mixin.scss";  
@import "reset.css";
```

Sass

```
@import "library"  
@import "mixins/mixin.less"  
@import "reset.css"
```

Less

```
@import "library"  
@import "mixins/mixin.styl"  
@import "reset.css"
```

Stylus

Mixins

```
@mixin bordered($width) {  
  border: $width solid #ddd;  
  
  &:hover {  
    border-color: #999;  
  }  
}  
  
h1 {  
  @include bordered(5px);  
}
```

Sass

```
.bordered (@width) {  
  border: @width solid #ddd;  
  
  &:hover {  
    border-color: #999;  
  }  
}  
  
h1 {  
  .bordered(5px);  
}
```

Less

```
bordered(w)  
  border: n solid #ddd  
  &:hover  
    border-color: #999  
  
h1  
  bordered(5px)
```

Stylus

```
h1 { border: 5px solid #ddd; }  
h1:hover { border-color: #999; }
```

Css

Mixins allow reuse of a group of CSS properties.

Functions



```
@function double($value) {  
  @return $value * 2;  
}
```

```
.box {  
  width: double(50px);  
}
```

CSS Best Practices and Optimization

Use Simple Selectors

Use simple selectors when possible. Complex selectors increase the parsing time.

Bad Example

```
body #navlist ul li a.button:hover {  
    background-color: blue;  
}
```

Better Example

```
.button:hover {  
    background-color: blue;  
}
```

Avoid Universal Selector for Styling

Avoid the universal selector (*) when not strictly necessary. The universal selector (*) affects every element and can slow down page rendering.

```
* {  
  margin: 0;  
  padding: 0;  
  font-size: 16px;  
}
```

Avoid Inline Styles

Avoid inline styles when not necessary. Inline styles make your HTML heavier and are harder to manage.

Bad Example

```
<div style="color: red; font-size: 18px;">Hello</div>
```

```
<p style="color: blue; font-size: 16px;">Test</p>
```

Avoid @import

- Avoid using @import for loading external CSS, as it delays stylesheet loading.
- Add external CSS with the <link> tag in the head section, so it loads before the page is rendered.

Example

```
<link rel="stylesheet" href="style.css">
```

Use Shorthand Properties

Use shorthand properties when possible. It saves space and is faster to parse.

Example:

```
/* Long version */  
margin-top: 10px;  
margin-right: 20px;  
margin-bottom: 10px;  
margin-left: 20px;
```

```
/* Shorthand version */  
margin: 10px 20px;
```

Avoid Redundant Code

Bad Practice

```
.box1 { margin: 10px; }
```

```
.box2 { margin: 10px; }
```

Good Practice

```
.box {  
    margin: 10px;  
}
```

Use Meaningful Class Names

Bad Practice

```
.a { }
```

```
.b { }
```

Good Practice

```
.navbar { }
```

```
.card { }
```

Use Properties that Not Cause Repaint of Animations

- Animation performance relies also on what properties you are animating.
- Some properties (like width, height, left, top), trigger a layout recalculation when animated, and should be avoided.
- If possible, use animation properties that do not cause repaint, like transforms, opacity and filter.

Combine and Minify CSS

Use one CSS file when possible, and remove spaces and comments to reduce file size.

You can use tools like:

- CSS Minifier
- PostCSS
- Online compressors

Any Questions?